Unit-2

Language Basics

- GUI & Console Application
- Identifier & Keywords
- Comments
- Variables, literals
- Data Types
- Expression
- Operator
  - Arithmetic Operator
  - Relational Operator
  - Logical Operator
  - Bitwise Operator
  - Assignment Operator
  - Miscellaneous Operator
- Statement
  - Declarative Statement
  - Expression Statement
- Namespace

Variables:
are the names you give to computer memory locations which are used to store values in a computer program.

Syntax:-

<datatype> <variable_name>;
<datatype> <variable_name> = <initial_value>;

Literals:
Literals are fixed values written directly in a C# program to represent constant data.

Expression
is a combination of operands (variables, literals, method calls) and operators that can be evaluated to a single value

Statement
a basic unit of execution of a program

Identifier & Keywords

Identifiers

are the names you use to identify variables, methods, classes, and other entities in your code.

Rules:

- Must start with a letter or an underscore (_).
- Can contain letters, digits, and underscores.
- Cannot be a C# keyword.
- Are case-sensitive.

Keywords

Keywords are reserved words in C# that have a specific meaning to the compiler.

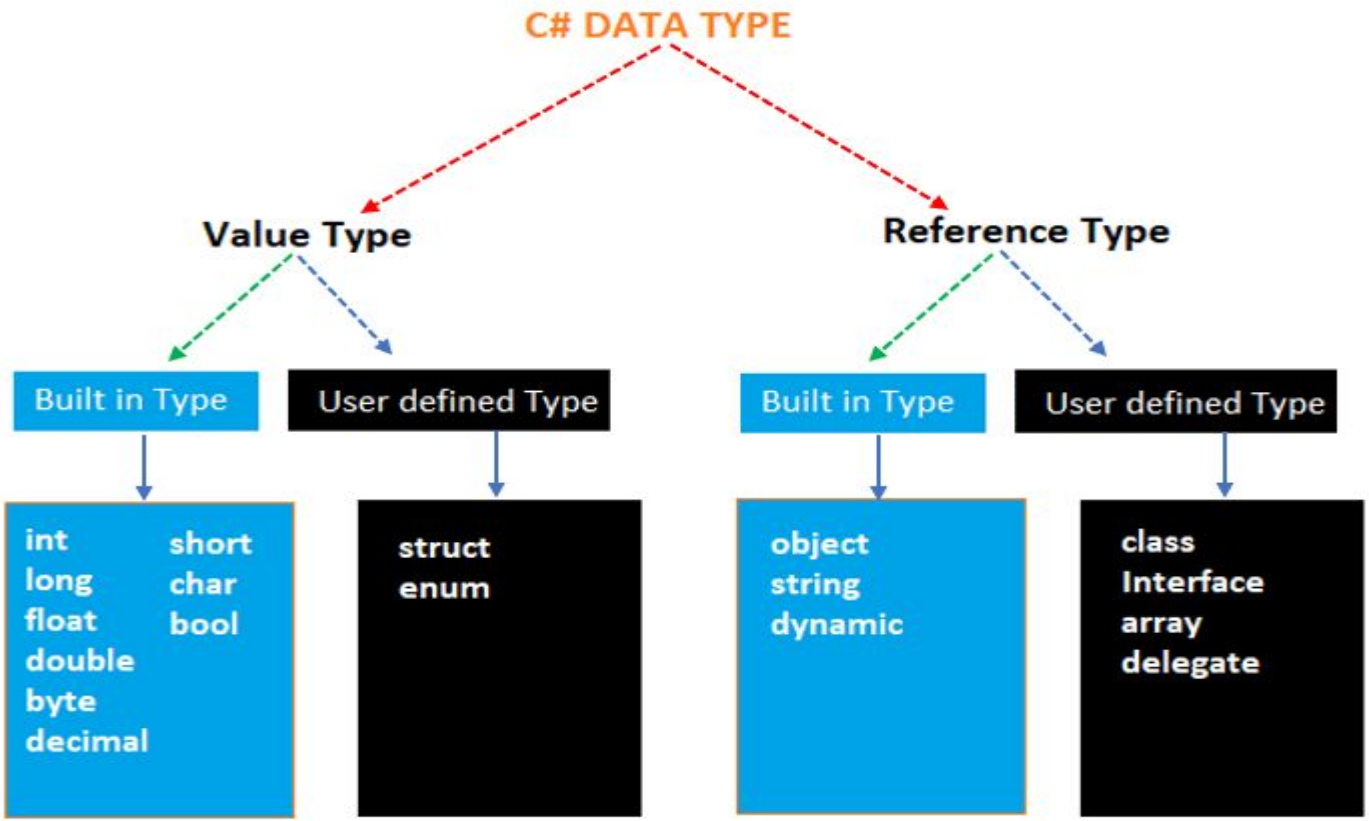
Examples of Keywords:

- Control flow: if, else, switch, case, for, while, do, break, continue, return
- Data types: int, float, double, char, string, bool
- Modifiers: public, private, protected, static, readonly, const
- Exception handling: try, catch, finally, throw
- Others: class, namespace, using, new, this, base

Comments

are used to explain and document the code

The compiler does not execute them and is purely for the benefit of developers

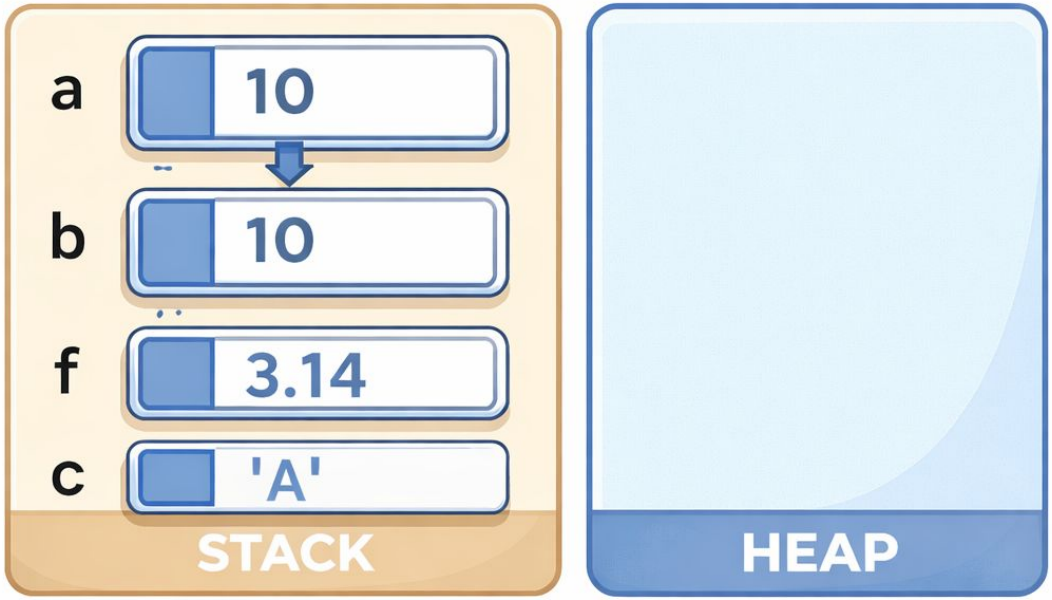


https://www.shekhali.com/

VALUE TYPE vs REFERENCE TYPE

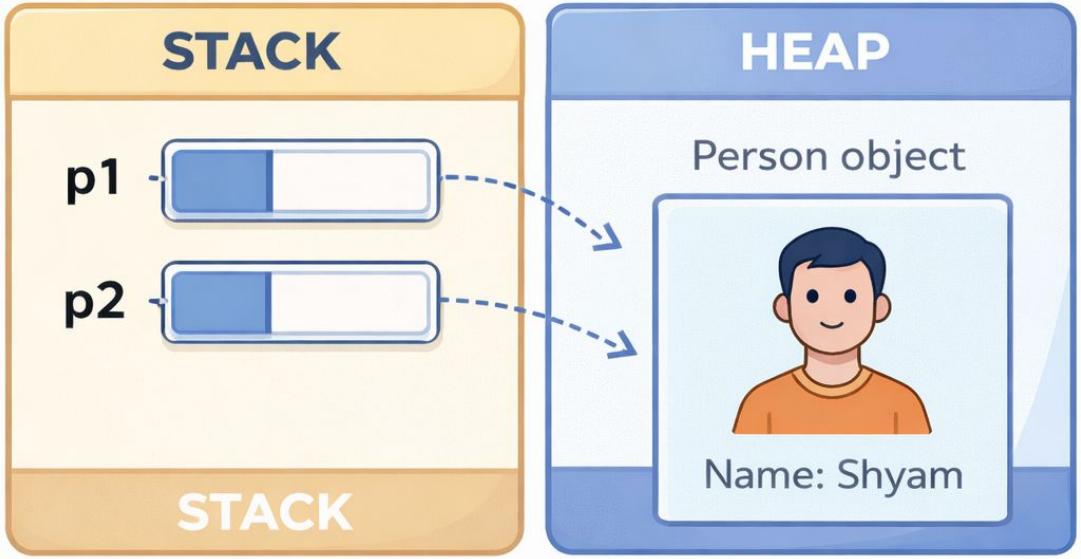
VALUE TYPE	REFERENCE TYPE
Stores the **value** itself	Stores a **reference**
Stored in **stack**	Stored in **heap**
Assignment copies the **value**	Assignment copies the **reference**
Changing one does **not affect** other	Changing one does **affect** others
int, double, bool	class, string, array

VALUE TYPE




```
int a = 10;
int b = a;
float f = 3.14;
char c = 'A;
```

REFERENCE TYPE




```
class Person
{ public string Name;
Person p1 = new Person();
p1.Name = 'Ram;
Person p2 = p1;
```

Value vs Reference Type

- Where it stores the data ?
- How it stores the data?
- How assignment/copy works?
- How does it behave after assignment/copy?

Integral Types (whole numbers)

value types that represent whole numbers without any fractional component

Aspect	int16 (short)	int32 (int)	int64 (long)
Bit Size	16 bits	32 bits	64 bits
Storage Size	2 bytes	4 bytes	8 bytes
Range (Signed)	-32,768 to 32,767	-2,147,483,648 to 2,147,483,647	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Range (Unsigned)	0 to 65,535	0 to 4,294,967,295	0 to 18,446,744,073,709,551,615
Use Cases	Memory-constrained environments	General-purpose integer arithmetic	Large numbers, high-precision calculations
Performance	Faster in some low-level operations due to smaller size	Balance of size and range, most commonly used	Slower due to larger size, used when needed for large values
Common Data Types	`short`, `ushort`	`int`, `uint`	`long`, `ulong`

Signed Integral

Signed integral types can represent both positive and negative numbers. They include sbyte, short, int, and long.

UnSigned Integral

Unsigned integral types can represent only non-negative numbers(positive) They include byte, ushort, uint, and ulong

Type	Range	Size	Default Value
`byte`	0 to 255	1 byte	0
`sbyte`	-128 to 127	1 byte	0
`short`	-32,768 to 32,767	2 bytes	0
`ushort`	0 to 65,535	2 bytes	0
`int`	-2,147,483,648 to 2,147,483,647	4 bytes	0
`uint`	0 to 4,294,967,295	4 bytes	0
`long`	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	8 bytes	0
`ulong`	0 to 18,446,744,073,709,551,615	8 bytes	0

Floating Types (fractional numbers)

- float (Single Precision):
  - **Size**: 4 bytes (32 bits).
  - **Range**: Approximately $\pm 1.5 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$.
  - **Precision**: Up to 7 significant digits.
  - **Suffix**: Use f or F to indicate a literal as float

- double (Double Precision):
  - **Size**: 8 bytes (64 bits).
  - **Range**: Approximately $\pm 5.0 \times 10^{-324}$ to $\pm 1.7 \times 10^{308}$.
  - **Precision**: Up to 15-16 significant digits.
  - **Suffix**: Use d or D to indicate a literal as double

- decimal (High Precision for Financial Calculations):
  - **Size**: 16 bytes. (128 bit)
  - **Precision**: Up to 28-29 significant digits.
  - **Range**: $\pm 1.0 \times 10^{-28}$ to $\pm 7.9 \times 10^{28}$.
  - **Suffix**: Use m or M to indicate a literal as decimal

Floating Point Precision

Floating point precision refers to the accuracy with which numbers are represented in computers, particularly when those numbers have decimal places.

Float Type (32-bit) Representation:

Total Bits: 32 bits

Sign bit: 1 bit

Exponent: 8 bits

Significand (Mantissa): 23 bits

Details:

- **1 bit for the **sign** (positive or negative).**

- **8 bits for the **exponent** (biased by 127).**

- **23 bits for the **significand** (Mantissa),**

Precision: 6-9 significant digits

Range: ±1.5 × 10^−45 to ±3.4 × 10^38

Double Type (64-bit) Representation:

Total Bits: 64 bits

Sign bit: 1 bit

Exponent: 11 bits

Significand (Mantissa): 52 bits

Details:

- **1 bit for the **sign** (positive or negative).**

- **11 bits for the **exponent** (biased by 1023).**

- **52 bits for the **significand** (Mantissa), which provides high precision (15-16 significant digits).**

Precision: 15-16 significant digits

Range: ±5.0 × 10^−324 to ±1.7 × 10^308

Decimal Type (128-bit) Representation:

Total Bits: 128 bits

Sign bit: 1 bit

Exponent: 8 bits

Significand (Mantissa): 112 bits

Details:

- **1 bit for the **sign** (positive or negative).**

- **8 bits for the **exponent** (biased by 127).**

- **112 bits for the **significand** (Mantissa), which provides high precision (28-29 significant digits).**

Precision: 28-29 significant digits

Range: ±1.0 × 10^−28 to ±7.9 × 10^28

Integral vs floating Type

- What type of data?
- Types and sub-types
- How it store the data?
- Range based on storing mechanims

Char

The char data type in C# is used to represent a single Unicode character.

It is a 16-bit type, capable of storing a single character encoded in UTF-16 (which represents Unicode characters using one or two 16-bit code units).

Operations:

1. Comparison:
2. Conversion:

Enum

- is a user-defined data type that has a fixed set of distinct, constant & related values.
- useful for representing a collection of related values in a type-safe

Underlying value & Type

```
enum DaysOfWeeks {
    Sunday,
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday
};
```

```
enum DaysOfWeeks:int {
    Sunday=1,
    Monday=2,
    Tuesday=3,
    Wednesday=4,
    Thursday=5,
    Friday=6,
    Saturday=7
};
```

Class Task

1. Create an enum for days of the week and print today’s day using it.
2. Create an enum for traffic lights (Red, Yellow, Green) and display action based on selected value.
3. Create an enum and print the underlying datatype of it.

   (Enum.GetUnderlyingType(typeof(Status))

Struct

- A value type used to group related data into a single unit
- Encapsulates small sets of variables and optional methods
- Best suited for simple data models (e.g., Point, Date, Color)

```
struct User {
 public string firsName;
 public string lastName;

 public string fullName() {
   return firsName + lastName;
 }
}
```

Struct

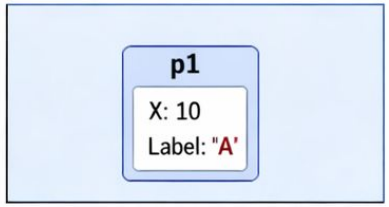
- **Type & storage:** Value type & stored in stack
- **Declaration:** ***struct*** keyword
- **Use:** for data models/DTO and short-lived objects
- **Pro:** Efficient memory usage in comparison to reference type(class)
- No inheritance
- No Default Constructor
- Supports Interface implementation

```
struct Point1 {
    public int X;
    public char Label;
}

struct Point2 {
    public int X;
    public String Name;
}

Class NameInfo{
     public String Name;
}
struct Point3 {
    public int X;
    public NameInfo Name;
}
```

```
struct Point
{
    public int X;
    public char Label;
}
```

1 Step 1: Create p1

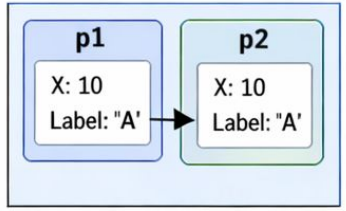
p1 = (X=10, Label='A')



- p1 is a value type stored on the stack.
- All fields in the struct are value types.

2 Step 2: Copy to p2

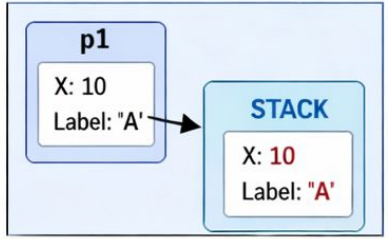
p2 = p1; // full copy



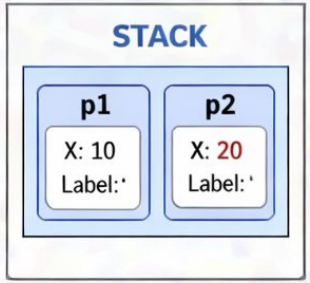
- p1 and p2 are independent.
- A full copy is made.

3 Step 3: Modify p2.x

p2.X = 20



- Value types are independent.
- Changing p2.X does not affect p1.X.

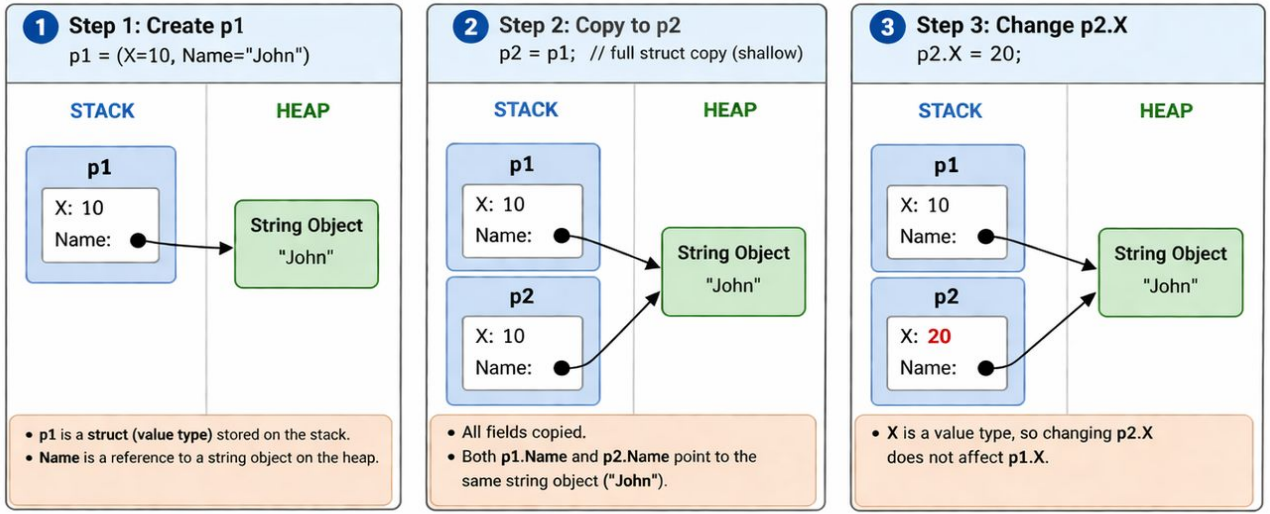


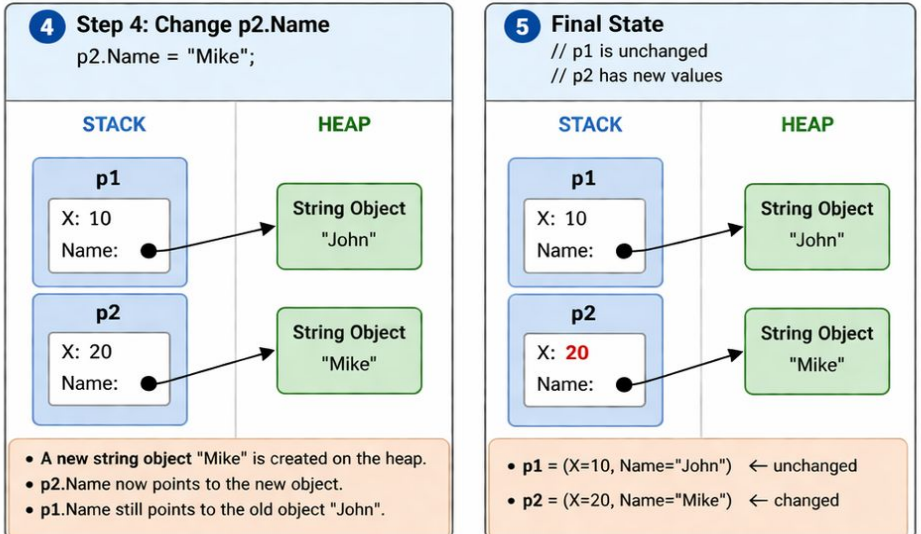
p1 = (X=10, Label='A') unchanged

p2 = (X=20, Label='A') changed

Key Takeaways

- Structs are copied by value.
- All fields in the struct are value types.
- Changing one struct's values does not affect the other.





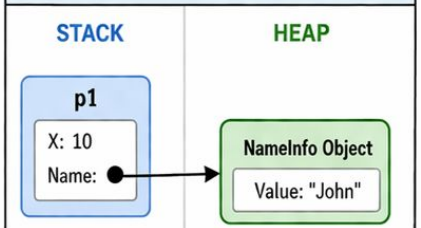
Code Summary

```
struct Person
{
   public int X;
   public string Name;
}

Person p1, p2;

p1 = (X: 10, Name: "John");
p2 = p1;      // copy

p2.X = 20;
p2.Name = "Mike";
```

```
class NameInfo
{
   public string Value;
}

struct Person
{
   public int X;
   public NameInfo Name;
}

Person p1, p2;
```

1 Step 1: Create p1

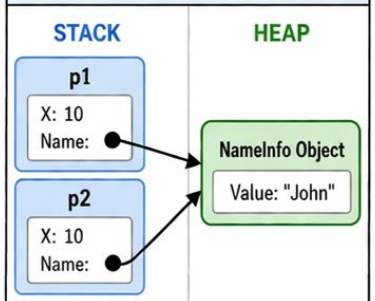
p1 = (X=10, Name.Value="John")



- **p1** is on the **stack.**
- **Name** points to a **NameInfo** object on the heap.

2 Step 2: Copy to p2

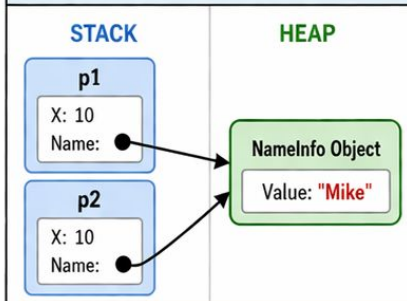
p2 = p1; // full struct copy (shallow)



- Both structs are independent.
- But both **Name** fields point to the **same object.**

3 Step 3: Modify object through p2

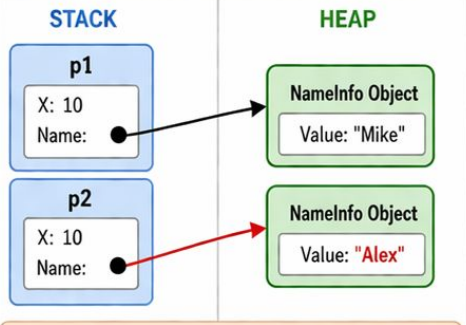
p2.Name.Value = "Mike";



- The shared object is modified.
- **Both p1** and **p2** see the **change!** (Value = "Mike")

4 Step 4: Reassign reference in p2

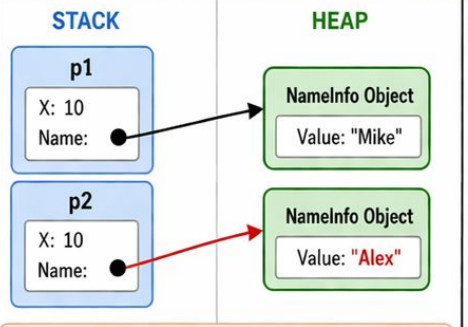
p2.Name = new NameInfo { Value = "Alex" };



- **p2** now points to a **new object.**
- **p1** still points to the **old object.**

5 Final State

p1 is unaffected by reassignment of p2.Name.



- **p1** = (X=10, Name.Value="Mike")
- **p2** = (X=10, Name.Value="Alex")

Key Takeaways

- Structs are **copied by value.**
- Reference fields inside a struct are copied (**same object**).
- **Modifying the object** affects all structs that share the reference.
- **Reassigning the reference** makes the structs independent again.

Class Task

- Write a C# program to copy one struct containing int and char to another and show that changes in the copied struct do not affect the original.
- Write a C# program to copy a struct containing an int and a reference type (class) and show that modifying the reference affects both structs while value type does not.

object

object is the base type of all types in C#.

statically typed and Type checking happens at compile time.

```
object x = "Hello";
// Compile-time error if invalid
Console.WriteLine(x.Length); // ❌ Error
// Must cast
Console.WriteLine(((string)x).Length); // ✅ Works
```

dynamic

bypasses compile-time checking

Type is resolved at runtime

```
dynamic x = "Hello";

// No compile-time error
Console.WriteLine(x.Length); // ✅ Works

dynamic x = 10;
Console.WriteLine(x.Length); // ❌ Runtime error
```

String

In C#, a string is a sequence of characters and is represented by the System.String class.

Concatenation:-

Concatenation is the process of joining two or more strings together.

In C#, this can be done using the + operator or the string.Concat() method.

Interpolation:-

String interpolation is a feature that allows you to embed expressions, variable, methods with return type, inside a constant string using the $ symbol.

```
string name = "Alice";
int age = 25;
string message = $"Name: {name}, Age: {age}";  // "Name: Alice, Age: 25"
```

Common Properties/Methods in String

- Length

```
int length = message.Length;
```

- Substring

```
string sub = message.Substring(2, 6); // "Hello"
```

- Replacing Text

```
string newMessage = message.Replace("World", "C#");
```

- Splitting a String

```
string csv = "apple,banana,grape";
string[] fruits = csv.Split(',');
```

- Checking if a String Contains Another String

```
bool containsWorld = message.Contains("World"); // true
```

- Upper and Lower Case

```
string upper = message.ToUpper();
string lower = message.ToLower();
```

- Trimming Whitespace

```
string text = "    C# Strings    ";
string trimmed = text.Trim(); // "C# Strings"
```

Class Task

Identify the longest word in a sentence.

Steps:-

- Ask String from console [Console.Readline()]
- Split sentence by ‘ ’ to find list of words
- Loop through words to find length of each words
- Words with length the biggest is the longest word

Find middle two character from your name using subString();

Ex: Sagar -> ga -> subString(2,2)

Reetika -> ti -> subString(3,2)

Find middle two chartcter from a dynamic string by finding middle

Type Conversion

process of converting a value from one data type to another.

Implicit Conversion:-

- Automatic type conversion
- No data loss occurs
- No special syntax needed.
- Smaller to larger data type & compatible data type
- Mostly Unidirectional(One way conversion)

Explicit Conversion(Type Casting):-

- Manual conversion
- Possible data loss due to truncation or overflow
- Requires explicit casting syntax
- Larger to smaller data type or vice-versa , compatible/incompatible types.
- Bidirectional

Implicit Conversion

- **Within the Same Data Type**

  smaller numeric type to a larger numeric type.

  Smaller floating type to larger floating type

  byte → short → int → long

  float->double -> long

```
short smallNum = 100;
int intNum = smallNum;    // Implicit conversion (short → int)
long longNum = intNum;    // Implicit conversion (int → long)
Console.WriteLine(longNum);  // Output : 100
```

Across Different Data Types

converting integers to floating-point types

int → float → double

long → float → double

Char <-> int

Char -> byte ?

Derived Class to Base Class (Reference Type Conversion)

a derived class object is assigned to a base class reference.

```
class Animal { }
class Dog : Animal { }

Dog dog = new Dog();
Animal animal = dog;  // Implicit conversion (Dog → Animal)

Console.WriteLine(animal.GetType());  // Output: Dog
```

Explicit Conversion

- **Direct Casting(Using(Type))**

Used for compatible types, but may result in data loss if the values exceed the target type range.

double → int

float → int

long → short

char → byte

```
double d = 9.78;
int num = (int)d;  // Explicit cast required
Console.WriteLine(num);  // Output : 9 (decimal part lost)
```

- **Using Convert Class**

provides safer methods for explicit conversion, handling different types properly. range.

string → int: Convert.ToInt32("123") -> 123

bool → int: Convert.ToInt32(true) → 1

object → int: Convert.ToInt32(obj) -> 1

```
string str = "123";
int num = Convert.ToInt32(str);
Console.WriteLine(num);  // Output: 123
```

- **Using Parse and tryParse()**

Used mainly for string to numeric conversions.

```
double num = double.Parse(str);
int num = int.Parse(str);
DateTime date = DateTime.Parse(str);
bool flag = bool.Parse(str);
object  enum = Enum.Parse(typeof(userDefinedEnum),str)
```

Class Task

1. Ask two number input from user using console.Readline, Type Cast them into numeric value and perform addition, subtraction, multiplication and division.
2. Ask a floating number input from user using Console.Readline, Type cast it into double first and then into byte;
3. Ask a single char from user using console.Readline, Type Cast it into char first and then into int

Out keyword

used to pass arguments to methods by reference.

No need to initialize before passing.

Used when a method needs to return multiple values.

It allows a method to modify the value of a parameter and pass it back to the caller

Statement

A statement in C# is a single line of code that performs an action.

- **Declaration Statement**: Declares variables
- **Expression Statement:** Performs calculations, method calls, assignments
- **Selection Statement:** (if, switch): Executes code based on conditions
- **Iteration Statement:**(for, while, do-while, foreach): Repeats code execution
- **Jump** (break, continue, return, goto): Alters execution flow
- **Using:** Manages resources efficiently

- **Declaration Statement**

Used to declare and initialize variables.

```
int age = 25;
string name = "Alice";
```

- **Expression Statement**

An expression that performs an operation (e.g., method calls, assignments, and object creation).

```
Console.WriteLine("Hello, World!"); // Method call
age = age + 1;                      // Assignment
name= "Sagar" + "Poudel";
var class1 = Class1();
```

- **Selection Statement(Conditional)**

Used to execute code based on conditions.

```
if-else statement
If-elseif-else statement
Switch statement
```

- **Iteration Statement(Loop Statement)**

Used to repeat a block of code.

```
for loop (Before Execution check)
foreach loop (Implicity Check)
while loop (Before Execution Check)
do-while loop (After Execution Check)
```

- **Jump Statement**

Alter the normal execution flow of the program

```
break
continue
return
goto
```

- **Using Statement**

primarily used for automatic resource management, ensuring that objects implementing IDisposable (like files, database connections, or network streams) are properly disposed of after use.

```
using (SqlConnection conn = new SqlConnection("your_connection_string"))
{
    conn.Open();
    using (SqlCommand cmd = new SqlCommand("SELECT * FROM Users", conn))
    using (SqlDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            Console.WriteLine(reader["Username"]);
        }
    }
} // Connection and Reader are automatically closed
```

Class Task(Conditional Statement)

1. WAP to ask 3 numeric input from user and find the largest among them.
    a. If two or more numbers are the same and also the largest, print "Two or more numbers are equal and the largest." instead of a single largest number.
    b. If all three numbers are equal, print "All numbers are the same."
    c. If the user enters non-numeric input, prompt them again until they provide a valid number.
2. Ask the user to enter their exam score (0-100). [Numeric value]
    a. Use an if-else if statement to assign
        i. grades: 90-100 → "Grade: A"
        ii. 80-89 → "Grade: B"
        iii. 70-79 → "Grade: C"
        iv. 60-69 → "Grade: D"
        v. Below 60 → "Grade: F"
3. WAP to ask a numeric input from user, between 1-7, Perform type Casting into int, use switch statement to print day of week based on that input.

Class Task(loop Statement)

1. Write a program that generates the multiplication table of a number provided by the user using a while or for loop. For example, if the user enters 5, the program should display the multiplication table from 5 x 1 to 5 x 10.
2. **Sum of Numbers (Using for and continue)** WAP to ask a positive numeric input from user between 1-10, Perform type Casting into int, Calculate the sum of numbers from 1 to N, excluding 5 (use continue).
3. WAP to ask a numeric input from user between 1-20, apply a for loop from 1 to 20
   - If the current loop number matches the user's input, skip it using **continue** and display a message: "Skipping number X".
   - If the loop reaches 15, terminate it early using break, displaying "Loop stopped at 15". Otherwise, print all other numbers normally.
4. Write a program that calculates the sum of all odd numbers from 1 to 200 using a while loop.

```
Initialize sum = 0
Initialize num = 1

While num <= 200:
    If num is odd (num % 2 != 0):
        Add num to sum
    Increment num by 1

Print the value of sum
```

Class Task(Optional)

1. User Login Simulation (Using while and break)
    - Ask the user for a password.
    - Allow up to 3 attempts using a while loop.
    - If the user enters the correct password ("secret123"), print "Access granted!" and exit the loop using break.
    - If they exceed 3 attempts, print "Access denied!".

Data Structure

way of organizing, storing, and managing data so it can be used efficiently.

We need data structures to choose the most efficient way to store, organize, and process data for a given task.

Choose data Structure based on:-

- how much data we have
- how often data changes
- how fast we want to access data

Types of Data Structure

1. Linear Data Structure

A linear list is a collection of elements arranged in a linear or sequential order.[one after another]

Example:-

- Array/Linear List
- Linked List
- Stack
- Queue

1. **Non-linear Data Structure**

is one where elements are not arranged in a simple sequential order. [hierarchical or network-like structure]

Example:-

- Tree
- Graph

Contiguous memory

means data is stored in continuous (adjacent) memory locations.

Address	Value
100	10
101	20
102	30
103	40

Non-Contiguous memory

means data is stored in different (schattered) memory locations.

Address	Value	Next
100	10	500
500	20	300
300	30	NULL

Array

Reference data type or linear data structure which stores collection of elements, all of the same type, stored in contiguous memory locations.

Arrays have a fixed size, which is specified at the time of creation and cannot be changed.

Type of Arrays:

- Single-Dimensional Array
- Multi-Dimensional Array
- Jagged Array (Array of Arrays)

Single Dimensional Array

The simplest type of array, resembling a list or a vector.

A single row of elements.

Declaration:

```
int[] arr = new int[5];

int[] numbers = { 1, 2, 3, 4, 5 };
```

Accessing:

```
Console.WriteLine(numbers[0]);
```

Multi-Dimensional (Rectangular, Tabular) Array

An array with two or more dimensions, resembling a matrix or a grid.

Rectangular shape, where each row has the same number of columns.

Declaration:

```
int[,] matrix = new int[3, 3];  // 3x3 matrix

int[,] matrix = { {1, 2, 3}, {4, 5, 6} }; // 3x3 matrix
```

Access:

matrix[1, 2] → 6

Jagged Array

An array where each element is another array, allowing for rows of varying lengths. Irregular shape, unlike multi-dimensional arrays.

Declaration:

```
int[][] jagged = new int[2][];

jagged[0] = new int[] { 1, 2 };

jagged[1] = new int[] { 3, 4, 5 };
```

Access:

matrix[1, 2] → 6

Class Task(Array)

1. WAP to create a singular array of size 10 of type int. Use for loop to ask 10 input from user and insert data in that array
2. WAP Create an singular array of string named hobbies and add/push any of your 5 hobbies into it. Ask all 5 Hobbies from Console.Readline(). [Use for loop]
    a. After completing the loop for 5 times. Ask a user if they want to add few more hobbies:
    b. If Yes:- Ask a number ie.
        i. How many more hobbies user want to add and repeat the process for that amount of time
        ii. If No:- Do Nothing
3. Create a multi dimensional array of int [10,10] and store multiplication table from 1 to 10 upto 10;

Class Task(Jagged Array)

1. Create a jagged array to store marks of 5 students. The catch is first 3 students has attempted 3 subject, 4th one 5 subject and last one 4 subject. Ask subjects marks from user
   a. Find average obtain of each student and store it in multidimensional array [Consider full marks as 100].
2. Create a jagged array of integers to store the quiz scores of 4 students:
   a. Student 1 attempted 2 quizzes.
   b. Student 2 attempted 4 quizzes.
   c. Student 3 attempted 3 quizzes.
   d. Student 4 attempted 5 quizzes.
3. Create a jagged array of strings to record the workouts of 5 friends over a week.
   Each friend logs the exercises they did each day.
   Number of exercises may vary each day.
   Display: Total exercises done by each friend over the week. The day with the most exercises for each friend

```
const int numFriends = 5;
const int numDays = 7;
// Create a jagged array:
5 friends -> each has 7 days -> each day has list of exercises
string[][][] workouts = new string[numFriends][numDays][];

for i->numFriends
 For j-> numDays
    exerciseCount=readline() //ask number of exercise
    for k->exerciseCount
        workouts[i][j][k]=readline()//ask exercise
```

Namespace

logical grouping of relevant & related types of data types [classes, interfaces, structs, enums, and delegates]

Container for organising codes and level of separation in a larger projects

Different Namespace

1. Default Namespace
2. User-Defined Namespace
3. User-Defined Hierarchical Namespace
4. User-Defined Nested Namespace

Using static

The using static directive allows you to import static members (methods, fields, properties, etc.) of a type directly into your code — so you can call them without specifying the class name.

```
using static System.Math;

class Program
{
    static void Main()
    {
        double result = Sqrt(16);  // 👈 No need to write Math.Sqrt
        Console.WriteLine(result); // Output: 4
    }
}
```

Class Task(Namespace)

1. Create a namespace named FitnessTracker.Workout
    a. Inside it, create a class WorkoutPlan.
    Add a method LogWorkout(string exercise) that prints: "Exercise Logged: [exercise]"
2. Create another namespace named FItnessTracker.User
    a. Inside it, create a class UserInfo.
    Add a method ShowProfile(name and age) to display the user's name and age.
3. In the Main() method:
    a. Create a User object and call ShowProfile().
    Create a Workout object and call LogWorkout() a few times with different exercises.

Characteristics & Features of Namespaces in C#

1. Code Organization via logical grouping
    a. Groups related classes and types together
    b. Makes code organized and readable
2. Hierarchical Structure
    a. Namespaces can be nested (like folders)
    b. namespace Company.Project.Module
3. Access with using Keyword
4. Reusability & Modularity
5. Namespaces are a compile-time feature

Ref keyword in C#[Pass by Reference]

It allows you to pass arguments by reference rather than by value.

This means the called method can modify the actual variable passed in, not just a copy of it.

```
void Double(ref int number)
{
   number = number * 2;
}

int x = 5;
Double(ref x);
Console.WriteLine(x); // Output: 10
```

- ref must be used **in both** the method signature and the method call.
- The variable **must be initialized** before it is passed in.

```
class Person
{
   public string Name;
}

void ChangePerson(Person p)
{
   p = new Person();
   p.Name = "Changed";
}

Person person = new Person();
person.Name = "Original";

ChangePerson(person);
Console.WriteLine(person.Name); // 👉 Still "Original"

void ChangePerson(ref Person p)
{
   p = new Person();
   p.Name = "Changed";
}

ChangePerson(ref person);
Console.WriteLine(person.Name); // 👉 "Changed"
```

Lab work Instruction

- Only Handwritten are accepted.
- Must contain screenshot of output of respective program attached within the each lab work you submit.
- Please don’t miss the deadline [Lab work submitted after deadline won’t be checked].